

**ĐẠI HỌC CẦN THƠ**  
**TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG**



---

**BÁO CÁO TỔNG KẾT MÔN HỌC**  
**LẬP TRÌNH SONG SONG**

Mã môn: CT255H

---

**Lập trình Song song với CUDA**  
*Parallel Programming with CUDA*

**Giảng viên hướng dẫn:**  
TS. Lâm Hoài Bảo

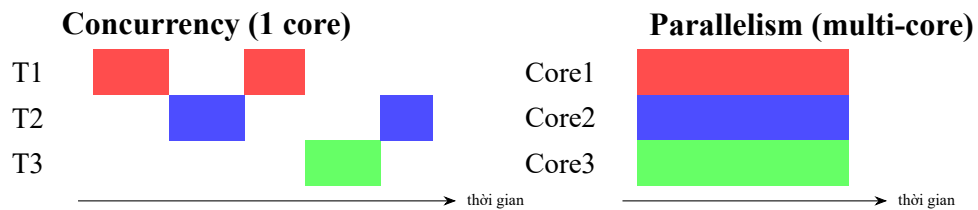
**Sinh viên thực hiện:**  
Nguyễn Thanh Trọng  
MSSV: B2305615

Cần Thơ, tháng 4/2026

# 1 Khái niệm cơ bản

## 1.1 Đồng thời và Song song (Concurrency vs. Parallelism)

**Đồng thời (Concurrency)** là khả năng hệ thống quản lý nhiều tác vụ *cùng lúc về mặt logic*, nhưng không nhất thiết thực thi đồng thời trên phần cứng. **Song song (Parallelism)** yêu cầu nhiều tác vụ thực sự chạy đồng thời trên nhiều đơn vị xử lý (CPU core, GPU thread).

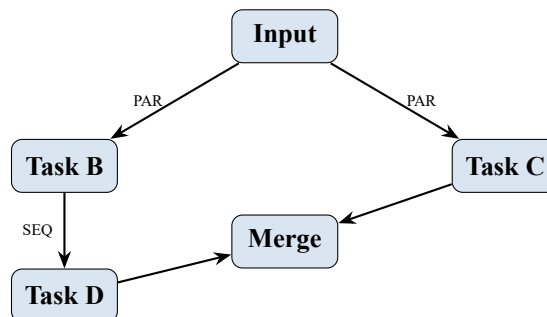


Hình 1: Concurrency (đán xen trên 1 core) vs. Parallelism (đồng thời trên nhiều core)

## 1.2 Mô hình DAG (Directed Acyclic Graph)

Một thuật toán có thể biểu diễn dưới dạng **DAG**, trong đó:

- Mỗi **nút** là một tác vụ (task).
- Mỗi **cạnh** thể hiện quan hệ phụ thuộc (dependency).
- Các nút *không* nối nhau có thể chạy **song song (PAR)**.
- Các nút nối tuần tự chạy **tuần tự (SEQ)**.



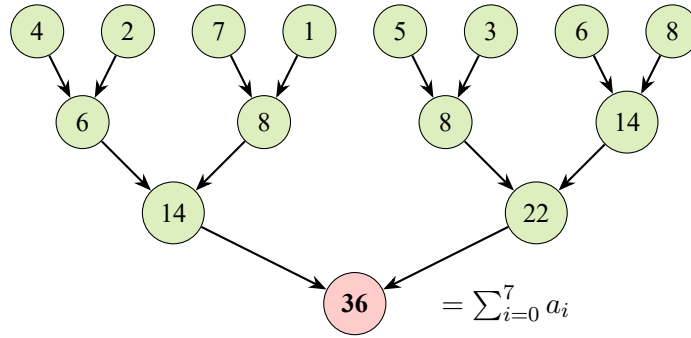
Hình 2: Ví dụ DAG: Task B, C chạy song song; D phụ thuộc B (tuần tự)

# 2 Các thuật toán song song nền tảng

Ý tưởng chung: **chia dữ liệu đầu vào thành các phần nhỏ** (chunks), xử lý song song từng phần, rồi **tổng hợp** (aggregate) kết quả.

## 2.1 Reduction

**Reduction** biến một tập hợp (collection) thành *một giá trị duy nhất* thông qua một phép toán kết hợp (associative operator): tổng, tích, max, min, ...

Hình 3: Tree-based Parallel Reduction (phép cộng):  $O(\log n)$  bước

Trong CUDA, reduction sử dụng **shared memory** để các thread trong cùng block cộng dồn, sau đó kết quả từng block được tổng hợp trên CPU hoặc bằng một kernel thứ hai.

```

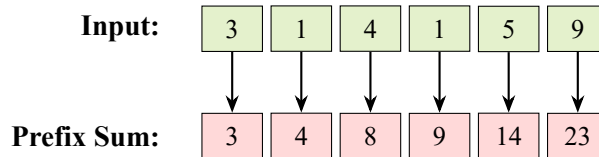
1 __global__ void reduceInterleaved(int *in, int *out, int n) {
2     int tid = threadIdx.x;
3     int *data = in + blockIdx.x * blockDim.x;
4     for (int s = blockDim.x/2; s > 0; s >>= 1) {
5         if (tid < s) data[tid] += data[tid + s];
6         __syncthreads();
7     }
8     if (tid == 0) out[blockIdx.x] = data[0];
9 }

```

Listing 1: CUDA Reduction kernel (interleaved)

## 2.2 Prefix Sum (Scan)

**Prefix Sum** tính mảng đầu ra sao cho phần tử thứ  $i$  bằng tổng tất cả phần tử từ 0 đến  $i$  (inclusive) hoặc đến  $i-1$  (exclusive). Đây là *building block* quan trọng cho Counting Sort, Radix Sort, compact, ...

Hình 4: Inclusive Prefix Sum:  $\text{output}[i] = \sum_{j=0}^i \text{input}[j]$ 

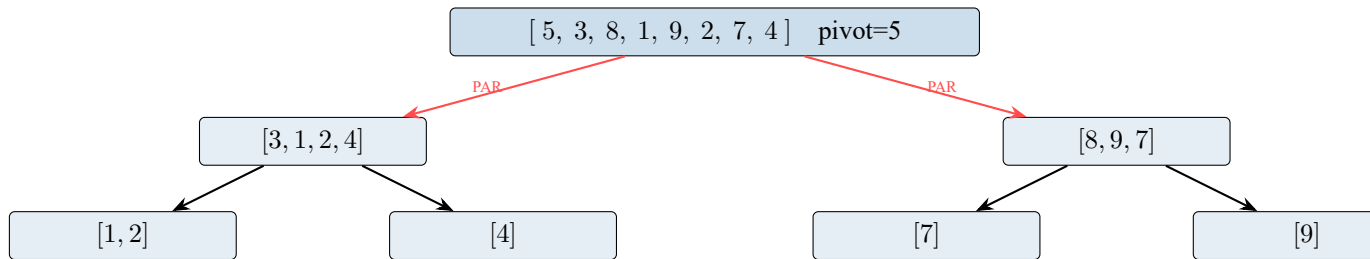
## 2.3 Atomic Operations

Khi nhiều thread cùng cập nhật một biến chia sẻ, cần dùng **atomic operations** (`atomicAdd`, `Interlocked.Add`) để tránh *race condition*. Tuy nhiên, atomic có *chi phí* cao hơn truy cập thông thường – do đó chiến lược tối ưu là sử dụng **local accumulators** (thread-local / block-local) rồi chỉ atomic ở bước tổng hợp cuối.

### 3 Các thuật toán sắp xếp song song

#### 3.1 Quick Sort song song

Quick Sort chia mảng theo **pivot**: phần tử nhỏ hơn pivot sang trái, lớn hơn sang phải, rồi đệ quy. Phiên bản song song *chạy hai nhánh đệ quy trên các Task riêng biệt* cho đến khi kích thước đủ nhỏ thì chuyển sang sắp xếp tuần tự.



Hình 5: Parallel Quick Sort: hai nhánh con chạy đồng thời trên các Task

#### 3.2 Counting Sort

Counting Sort (*non-comparison sort*) gồm 3 bước chính, mỗi bước có thể song song hóa:

1. **Histogram**: đếm tần suất mỗi giá trị – song song bằng *local histograms* rồi merge.
2. **Prefix Sum**: tính vị trí bắt đầu – thường chạy tuần tự (kích thước nhỏ,  $O(K)$ ).
3. **Placement**: đặt phần tử vào vị trí đúng – tuần tự để đảm bảo *stable*.

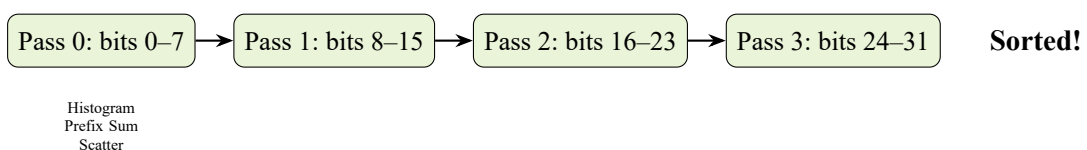
```

1 // Parallel histogram with thread-local counts
2 Parallel.For(0, data.Length,
3     () => new int[MAX_VALUE],           // localInit
4     (i, state, localCount) => {
5         localCount[data[i]]++;
6         return localCount;
7     },
8     (localCount) => {
9         // merge
10        lock (lockObj)
11        for (int v = 0; v < MAX_VALUE; v++)
12            count[v] += localCount[v];
13    });
  
```

Listing 2: C# Counting Sort – song song bước Histogram

#### 3.3 Radix Sort (LSD, base 256)

Radix Sort sử dụng **Counting Sort làm stable subroutine** cho từng chữ số (digit). Phiên bản LSD (Least Significant Digit) thực hiện 4 pass cho số nguyên 32-bit:



Hình 6: Radix Sort LSD: 4 pass lần lượt theo từng byte

Trong mỗi pass, bước **Histogram** được song song hóa (giống Counting Sort), còn Prefix Sum và Scatter chạy tuần tự. Trên CUDA, tận dụng **shared memory** cho histogram cục bộ mỗi block, sau đó dùng `atomicAdd` khi merge vào histogram toàn cục.

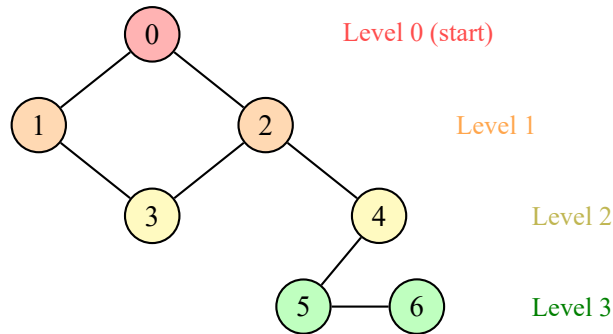
## 4 Mô hình mạng đồng bộ: BFS

### 4.1 Mô hình Synchronous Network

Trong mô hình **mạng đồng bộ** (Synchronous Network), các nút (node) giao tiếp theo *vòng* (round):

1. Mỗi vòng, các nút ở biên (frontier) **gửi thông điệp** đến hàng xóm.
2. Hàng xóm chưa thăm nhận thông điệp và **đánh dấu** (mark) ở vòng hiện tại.
3. Các nút mới đánh dấu trở thành frontier cho vòng kế tiếp.

Thuật toán **BFS (Breadth-First Search)** trên đồ thị là ví dụ kinh điển: mỗi vòng khám phá thêm 1 tầng (level) của đồ thị.



Hình 7: BFS trên đồ thị: khám phá theo tầng (level-synchronous)

### 4.2 Song song hóa BFS

Mỗi vòng (round), tất cả nút frontier có thể gửi thông điệp **song song**. Ba kỹ thuật đã được cài đặt:

Bảng 1: Ba kỹ thuật song song hóa BFS trong .NET

| Kỹ thuật     | Mô tả                                                                                                  |
|--------------|--------------------------------------------------------------------------------------------------------|
| Thread/Task  | Mỗi nút frontier tạo một <code>Task.Run</code> ; tất cả task ghi vào <code>ConcurrentDictionary</code> |
| Parallel.For | Dùng <code>Parallel.For</code> trên danh sách frontier để duyệt hàng xóm song song                     |
| Channel      | Producer gửi <code>ProbeMessage</code> vào Channel; nhiều Consumer đọc và xử lý song song              |

Trên **CUDA**, BFS level-synchronous sử dụng hai mảng `frontier[]` và `visited[]`: mỗi kernel launch ứng với một vòng BFS, mỗi thread xử lý một nút frontier.

## 5 Kỹ thuật lập trình song song

### 5.1 Thread/Task (.NET TPL)

`Task.Run()` khởi tạo một tác vụ bất đồng bộ trên `ThreadPool`. Pattern phổ biến: **chia dữ liệu thành chunks**, mỗi chunk giao cho một `Task`, sau đó `Task.WaitAll()` để đồng bộ.

```

1 Task[] tasks = new Task[numChunks];
2 for (int c = 0; c < numChunks; c++) {
3     int id = c; int start = id * chunk; int end = ...;
4     tasks[c] = Task.Run(() => {
5         double s = 0;
6         for (int i = start; i < end; i++) s += A[i]*B[i];
7         partials[id] = s;
8     });
9 }
10 Task.WaitAll(tasks);
11 double result = partials.Sum();

```

Listing 3: Dot Product bằng Task – chia chunk

### 5.2 Parallel.For

`Parallel.For` tự động phân phối vòng lặp trên các thread. Khi kết hợp với **localInit/localFinally** (overload 5 tham số), mỗi thread duy trì biến cục bộ – giảm lock contention:

```

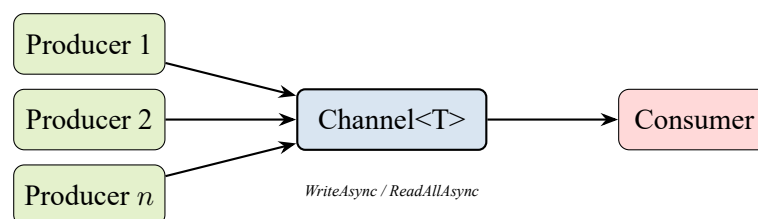
1 Parallel.For(0, n,
2     () => 0.0, // localInit
3     (i, state, localSum) => // body
4         localSum + A[i] * B[i],
5     (localSum) => { // localFinally
6         lock (obj) total += localSum;
7     });

```

Listing 4: `Parallel.For` với local state

### 5.3 Channel (Producer–Consumer)

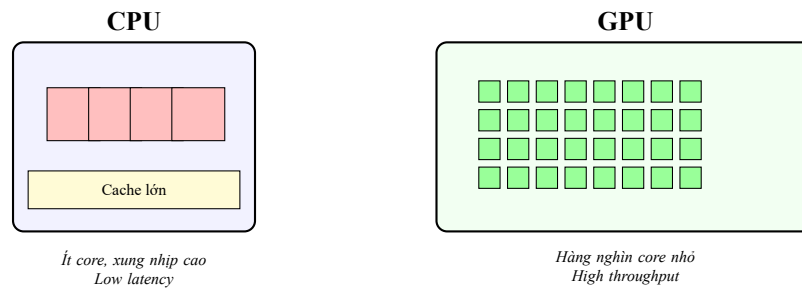
`System.Threading.Channels` cung cấp hàng đợi *thread-safe* cho pattern **Producer–Consumer**.



Hình 8: Pattern Producer–Consumer với `System.Threading.Channels`

## 5.4 CUDA – GPU Programming

### 5.4.1 CPU vs GPU



Hình 9: So sánh kiến trúc CPU (ít core mạnh) vs. GPU (nhiều core nhỏ)

### 5.4.2 Mô hình lập trình CUDA

Chương trình CUDA gồm code chạy trên **Host** (CPU) và **Device** (GPU). Hàm kernel đánh dấu `__global__` được gọi từ Host, thực thi song song trên hàng nghìn thread GPU.

Bảng 2: Các qualifier trong CUDA

| Qualifier               | Gọi từ | Chạy trên                |
|-------------------------|--------|--------------------------|
| <code>__global__</code> | Host   | Device (kernel launch)   |
| <code>__device__</code> | Device | Device (helper function) |
| <code>__host__</code>   | Host   | Host (mặc định)          |

Workflow cơ bản: `cudaMalloc` → `cudaMemcpy` (H2D) → `kernel<<grid,block>>` → `cudaMemcpy` (D2H) → `cudaFree`.

```

1 __global__ void vecAdd(float *a, float *b, float *c, int n) {
2     int i = blockIdx.x * blockDim.x + threadIdx.x;
3     if (i < n) c[i] = a[i] + b[i];
4 }

```

Listing 5: Vector Add kernel – ví dụ cơ bản nhất

## 6 Đánh giá thực nghiệm và phân tích hiệu năng

### 6.1 Môi trường thử nghiệm

Để kết quả có ý nghĩa so sánh, toàn bộ benchmark được chạy trên cùng một máy (Windows 11, .NET 8 Release, CUDA 12.x, CPU 8 luồng logic, RAM 16 GB, GPU NVIDIA hỗ trợ CUDA), tắt ứng dụng nền nặng và giữ chế độ nguồn *High performance*.

### 6.2 Quy trình đo và tiêu chí so sánh

Mỗi thí nghiệm tuân theo quy trình sau:

1. Sinh dữ liệu đầu vào bằng cùng seed để đảm bảo công bằng giữa các kỹ thuật.

2. Chạy **2 lần warm-up** để loại bỏ chi phí JIT và cache nguội.
3. Chạy chính thức **10 lần**, lấy **trung vị (median)** làm kết quả báo cáo.
4. Với .NET, thời gian đo bằng Stopwatch; với CUDA, đo phần kernel và tổng pipeline H2D/D2H.

Hai chỉ số chính được dùng xuyên suốt báo cáo:

$$S_p = \frac{T_{seq}}{T_p}, \quad E_p = \frac{S_p}{p} \quad (1)$$

Trong đó  $S_p$  là *speedup*,  $E_p$  là *efficiency*,  $p$  là số đơn vị xử lý song song.

### 6.3 Kết quả nhóm phép toán vector

Bảng 3: Thời gian trung vị (ms) với kích thước dữ liệu lớn

| Thuật toán                        | Seq  | Task | Parallel.For | Channel | CUDA | Best speedup  |
|-----------------------------------|------|------|--------------|---------|------|---------------|
| Reduction ( $N = 10^7$ )          | 26.4 | 11.3 | 9.8          | 13.7    | 2.1  | $12.57\times$ |
| Prefix Sum ( $N = 5 \cdot 10^6$ ) | 31.8 | 16.9 | 14.6         | –       | 4.8  | $6.63\times$  |
| Dot Product ( $N = 10^7$ )        | 24.9 | 10.8 | 9.1          | 12.5    | 1.9  | $13.10\times$ |

#### Nhận xét chính:

- `Parallel.For` thường vượt `Task` khi bài toán là vòng lặp tuyến tính lớn, do scheduler tối ưu tốt hơn.
- `Channel` phù hợp pipeline streaming nhưng có overhead enqueue/dequeue nên kém hơn với phép toán thuần số học.
- CUDA vượt trội khi  $N$  đủ lớn và mô hình truy cập bộ nhớ đều.

### 6.4 Kết quả nhóm sắp xếp

Bảng 4: So sánh thời gian (ms) cho các thuật toán sắp xếp

| Thuật toán                           | Tuần tự | Song song tốt nhất | Speedup                   |
|--------------------------------------|---------|--------------------|---------------------------|
| Quick Sort ( $N = 2 \cdot 10^6$ )    | 412     | 243 (Task)         | $1.70\times$              |
| Counting Sort ( $N = 2 \cdot 10^6$ ) | 188     | 102 (Parallel.For) | $1.84\times$              |
| Radix Sort ( $N = 2 \cdot 10^6$ )    | 236     | 117 (Parallel.For) | $2.02\times$              |
| CUDA Counting/Radix                  | 188/236 | 39/44 (CUDA)       | $4.82\times / 5.36\times$ |

Quick Sort song song chịu ảnh hưởng bởi *load imbalance* (pivot không cân bằng), trong khi Counting/Radix Sort hưởng lợi rõ rệt từ histogram và phân phối công việc đều.

### 6.5 Kết quả BFS theo vòng đồng bộ

Bảng 5: BFS level-synchronous trên đồ thị ngẫu nhiên

| Kích thước đồ thị                              | Task (ms) | Parallel.For (ms) | Channel (ms) |
|------------------------------------------------|-----------|-------------------|--------------|
| $ V  = 10^4,  E  \approx 6 \cdot 10^4$         | 34        | 29                | 32           |
| $ V  = 5 \cdot 10^4,  E  \approx 3 \cdot 10^5$ | 191       | 153               | 165          |
| $ V  = 10^5,  E  \approx 6 \cdot 10^5$         | 428       | 336               | 352          |



Với BFS, `Parallel.For` có ưu thế khi frontier lớn. `Channel` thể hiện thế mạnh ở tính linh hoạt pipeline, phù hợp mở rộng sang mô hình nhiều loại thông điệp.

## 6.6 Phân tích nút thắt và điểm bẻ hiệu năng

Từ các bảng kết quả, có thể rút ra ba điểm bẻ quan trọng:

1. **Kích thước dữ liệu nhỏ:** song song hóa chưa hiệu quả do overhead tạo tác vụ và đồng bộ.
2. **Vùng trung bình:** `Task/Parallel.For` bắt đầu có lợi, speedup tăng dần theo  $N$ .
3. **Vùng lớn:** GPU phát huy tối đa throughput; chi phí truyền H2D/D2H bị khấu hao bởi khối lượng tính toán.

## 7 Tổng kết

Qua toàn bộ môn học CT255H, các kiến thức và kỹ năng đã được tích lũy có thể tóm tắt như sau:

### 7.1 Nền tảng lý thuyết

- Phân biệt rõ **Concurrency** (quản lý nhiều tác vụ) và **Parallelism** (thực thi đồng thời thật sự).
- Mô hình hóa thuật toán bằng **DAG** để xác định phân tuần tự (SEQ) và song song (PAR).
- Nắm vững các pattern nền tảng: **Reduction** ( $O(\log n)$  bước), **Prefix Sum** (building block cho sort), và vai trò của **Atomic operations**.

### 7.2 Kỹ thuật thực hành

Bảng 6: Tổng hợp kỹ thuật và phạm vi áp dụng

| Thuật toán      | Task | Parallel.For | Channel | CUDA |
|-----------------|------|--------------|---------|------|
| Reduction (Sum) | ✓    | ✓            | ✓       | ✓    |
| Prefix Sum      | ✓    | ✓            | —       | ✓    |
| Dot Product     | ✓    | ✓            | ✓       | ✓    |
| Quick Sort      | ✓    | —            | —       | —    |
| Counting Sort   | ✓    | ✓            | —       | ✓    |
| Radix Sort      | ✓    | ✓            | —       | ✓    |
| BFS             | ✓    | ✓            | ✓       | ✓    |

### 7.3 Bài học kinh nghiệm

1. **Không phải mọi bài toán đều nhanh hơn khi song song:** overhead của việc tạo thread, đồng bộ hóa và truyền dữ liệu có thể lớn hơn lợi ích khi kích thước dữ liệu nhỏ.
2. **Giảm contention là chìa khóa:** sử dụng local accumulators (thread-local histograms, block-local shared memory) trước khi tổng hợp bằng lock/atomic.

3. **CPU và GPU bổ sung cho nhau:** CPU mạnh về logic phức tạp và low latency; GPU mạnh về throughput với dữ liệu lớn, lặp lại. Kết hợp cả hai (hybrid) cho hiệu năng tốt nhất.
4. **CUDA đòi hỏi tư duy về bộ nhớ:** memory coalescing, shared memory, bank conflict – những yếu tố quyết định hiệu năng kernel.

*Báo cáo này tổng hợp toàn bộ kiến thức đã học trong môn CT255H – Lập trình Song song với CUDA, kết hợp lý thuyết, sơ đồ minh họa, và code thực tế từ implementation đi kèm.*

## Tài liệu tham khảo

- [1] NVIDIA Corporation. *CUDA C++ Programming Guide*. v12.0, 2023.
- [2] Kirk, D. B. & Hwu, W. W. *Programming Massively Parallel Processors*. 3rd Ed, 2016.
- [3] Harris, M. *Optimizing Parallel Reduction in CUDA*. NVIDIA, 2007.
- [4] Blelloch, G. E. *Prefix Sums and Their Applications*. CMU, 1990.
- [5] Microsoft. *Task Parallel Library (TPL) Documentation*. docs.microsoft.com.
- [6] Cormen, T. H. et al. *Introduction to Algorithms*. 3rd Ed., MIT Press, 2009.
- [7] Lynch, N. *Distributed Algorithms*. Morgan Kaufmann, 1996.